flashjet

GPU jet clustering — benchmarking against FastJet

C. Gupta · 2026-08-17

What flashjet is

A GPU implementation of the standard sequential-recombination jet algorithms — anti-k_t, k_t, Cambridge–Aachen — written in Triton/PyTorch.

- Clusters a **batch of jets or events at once**, directly in GPU memory.
- The data **never moves back to the CPU** during clustering.
- Returns the particle→jet assignment **and the complete merge history** (the full binary tree of which pair merged, and at what distance).

Why the merge history matters: substructure observables — groomed mass, z_g, R_g, Lund coordinates, exclusive subjets — are then just **cheap reads of that tree**, with no second clustering pass.

> **The goal of this study:** is flashjet a viable **substitute for FastJet**?
> That needs two things — it must give the **same jets**, and it must be **faster**.

What was established before this study

The physics was validated; the speed was not.

what	how it was checked	outcome
the algorithms are correct	independent NumPy tree-walks; **129 unit tests**	pass
clustering matches FastJet	jet-by-jet kinematics tests	exact
works on real detector data	reclustered CMS jets vs the values CMS stored	p_T ratio **1.000000**
grooming is correct	our soft-drop mass vs CMS's	median Δ **−0.004 GeV**
substructure is correct	R_g, z_g vs CMS	$\Delta \leq 2\times10^{-4}$
the physics is right	soft-drop z_g, Lund plane, jet areas vs analytic predictions	reproduced

Small disagreements were traced and explained rather than ignored: they come from how the input file **stores** numbers (dropped soft particles, float rounding), not from the algorithm.

> **⇒ Correctness was settled. What was missing was a proper answer to "how much faster is it, on real physics data?"** — that is this study.

The benchmark

what · on what · how

What is being compared

Two clusterers, given **exactly the same jets**, asked to do **exactly the same job** (anti-k_t, same radius R):

	runs on	what it is
flashjet	**GPU**	this library
FastJet — "vectorised"	CPU	the standard tool, called through its efficient batched interface
FastJet — "per-jet"	CPU	the standard tool, called one jet at a time

> **All speedups quoted here are against the vectorised (faster) FastJet.**
> Comparing against the per-jet loop would make flashjet look ~5× better still, but that would not be a fair fight.

Two independent software environments are used so the two libraries cannot interfere with each other's dependencies.

What data

CMS Open Data — publicly released simulation, so every number here can be shown freely, with no approval needed.

process	jets used	average particles per jet
DY+jets	30 000	10.8
W+jets	30 000	15.1
$t\bar{t}$ (all-hadronic)	30 000	22.2
QCD (high-p_T dijets)	28 250	34.0

Why these four: they are ordinary physics processes, and together they span a **3× range in how busy a jet is** (11 → 34 particles). Jet clustering cost is driven by that number, so this range is what lets us see a **trend** rather than one number.

Format: MINIAOD, which is the tier that still contains the individual particles inside each jet. Read straight over the network from the public CERN storage.

How the measurement is done

1. Take the jets. Extract the particles belonging to each jet from the Open Data files, once, and save them.

2. Cluster with flashjet on the GPU. Time it, then **save the exact same input events to a file**.

3. Cluster those same saved events with FastJet on the CPU. Time it.

4. Check they agree. Compare the **number of jets found**, event by event. If the two disagree, the timing is meaningless.

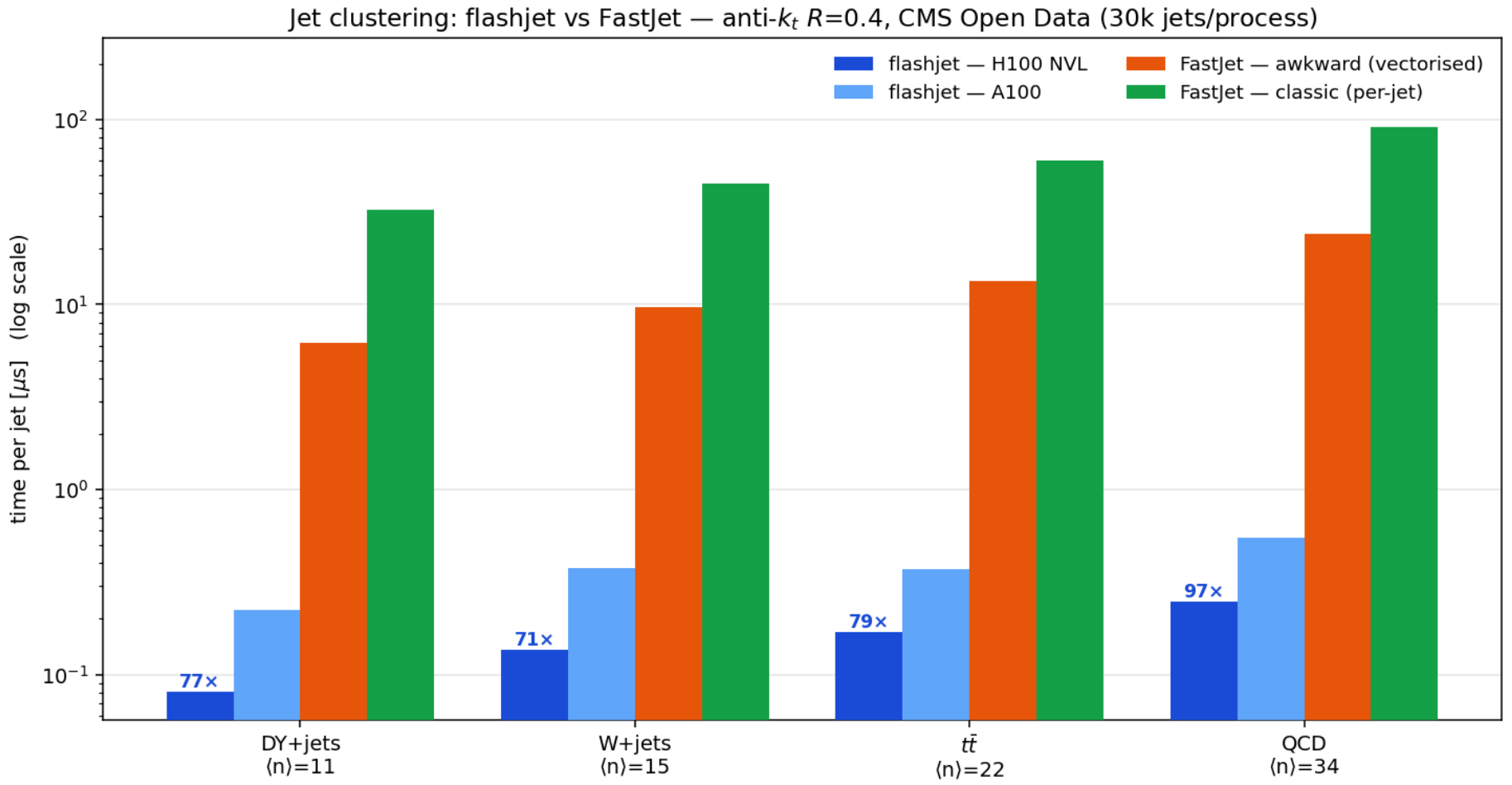
Repeated for **4 processes × 2 jet radii (R = 0.4 and 0.8) × 2 GPUs**.

> **The key discipline:** both clusterers read the **same saved file**. Nothing is regenerated in between, so this is a like-for-like comparison and not two separate measurements that happen to use the same dataset name.

Hardware: NVIDIA **A100** and **H100 NVL**, one full GPU each (no shared or partitioned cards). CPU baseline runs on the same machine.

Results

Time to cluster one jet



Anti-k_t, R=0.4. **Log scale** — each gridline is 10×. Blue = flashjet on GPU, orange/green = FastJet on CPU. Labels give the speedup over vectorised FastJet.

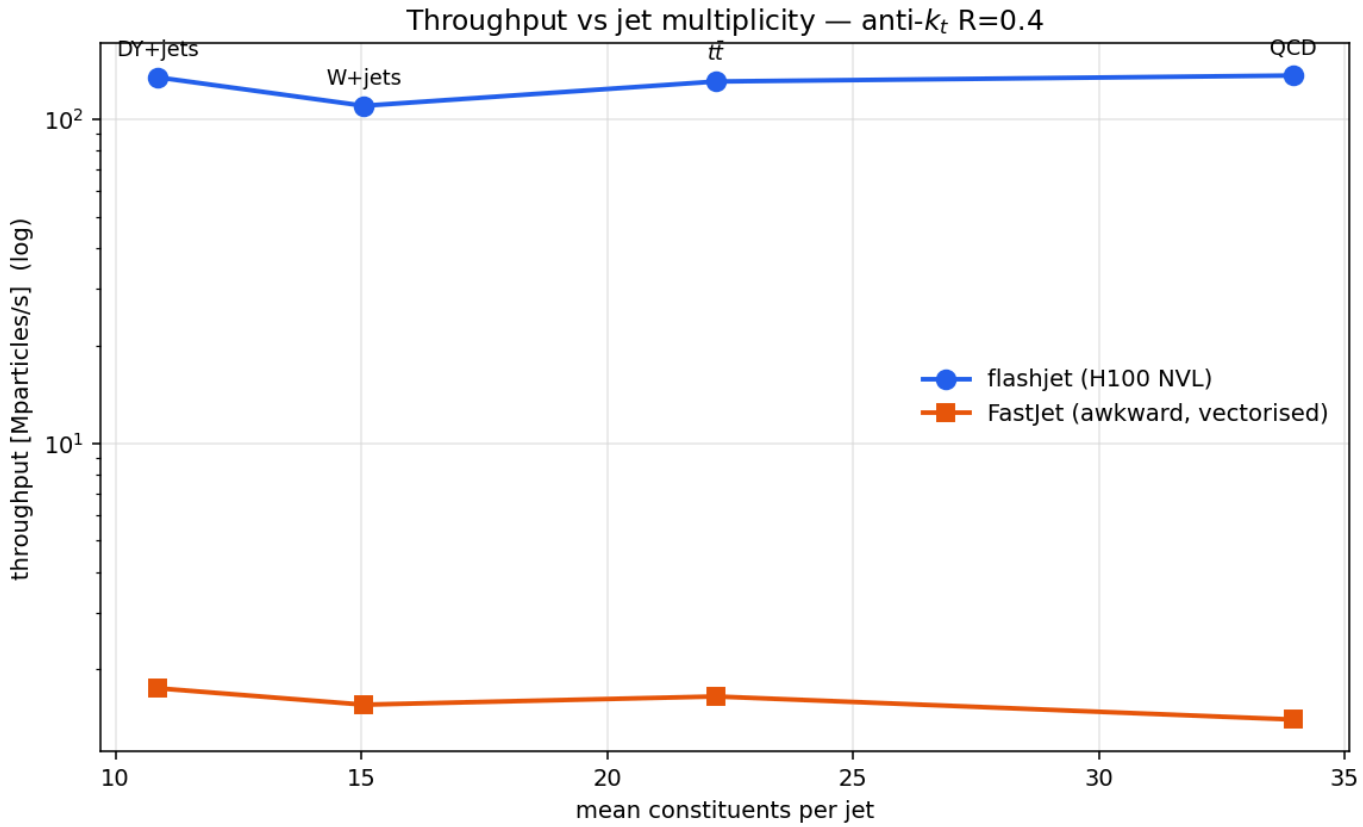
The numbers

Microseconds to cluster **one jet** (anti-k_t, R=0.4):

process	particles/jet	flashjet H100	flashjet A100	FastJet (vectorised)	FastJet (per-jet)	speedup
DY+jets	10.8	**0.081**	0.223	6.20	32.43	**77×**
W+jets	15.1	**0.137**	0.373	9.67	44.84	**71×**
$t\bar{t}$	22.2	**0.170**	0.373	13.46	59.95	**79×**
QCD	34.0	**0.249**	0.549	24.24	91.64	**97×**

- **Same jets:** number of jets found agrees with FastJet in **~100 %** of cases (7 of 8 runs exactly 100 %, one at 99.997 %).
- **Not a fluke of the jet radius:** R=0.8 gives the same picture (H100 65–96×).
- **Newer GPU helps:** A100 → H100 is a further ~2.2× on identical input.

The trend that matters



Throughput vs how busy the jet is. The four processes sit on **one common trend** — this is a property of the algorithm, not of any particular physics sample.

The busier the jet, the bigger the advantage (77× → 97×): FastJet's cost grows quickly with the number of particles, flashjet's grows much more slowly.

Where the time actually goes

We profiled the GPU to check *why* it is fast, and whether anything is wasted.

part of the code	share of GPU time
the clustering itself	**97.6 %**
extracting the jet assignment	1.5 %
everything else	< 1 %

Moving data between CPU and GPU is **negligible** once running.

> ⇒ **There is no hidden overhead.** Essentially all the time is the real work.
> It also means any *future* speedup has to come from that one piece of code.

A finding worth acting on

A deeper hardware-level profile shows flashjet is **not** limited by memory speed — it is limited by **not giving the GPU enough work at once**:

measure	H100	what it means
memory bandwidth used	**0.02 %**	memory is essentially idle
GPU "filled up"	**0.32×**	the GPU is **not filled even once**
occupancy achieved	**6.25 %**	most of the chip is unused

> **⇒ flashjet is already ~80× faster than FastJet while leaving most of a modern GPU idle.** The internal settings that decide how work is spread across the GPU were tuned on an older, smaller card and do not suit an H100.

This is good news: it is a concrete, identified path to going faster still, not an unexplained limit.

Conclusions

Conclusions

1. **flashjet gives the same jets as FastJet.**
   ~100 % agreement on jet counts across every process and radius tested,
   on top of the earlier jet-by-jet agreement with real detector data.

2. **flashjet is 65–97× faster** than the standard vectorised FastJet on a
   modern GPU — **0.08–0.25 µs per jet**.

3. **The advantage grows with jet complexity.** Busier jets → bigger speedup.
   This is the regime that matters most for boosted-object physics.

4. **The result is robust**, not tuned: four unrelated physics processes,
   two jet radii, two GPU generations, all consistent.

5. **There is still significant headroom.** The GPU is largely idle; the
   work-distribution settings were inherited from older hardware.

> **Bottom line: as a drop-in replacement for CPU jet clustering, flashjet is both correct and roughly two orders of magnitude faster.**

What is done, what is next

Checklist

Correctness — **complete**

✓ 129 unit tests pass, including GPU-only tests
✓ matches FastJet exactly in kinematics tests
✓ matches real detector-stored values jet-by-jet (p_T, groomed mass, R_g, z_g)
✓ reproduces analytic physics predictions
✓ same number of jets as FastJet in every timing run

Performance — **complete for the jet-clustering regime**

✓ 4 physics processes × 2 radii × 2 GPU generations, on public data
✓ profiled: no hidden overhead; bottleneck identified

Infrastructure — **complete**

✓ reproducible pipeline from public data to final plots
✓ results are freely presentable (no approval needed)

To do — in priority order

	item	why	effort
1	**Compare against C++ FastJet**	every number here uses FastJet's **Python** interface. The production tool is C++. Until we measure it, the honest caveat is *"some of this could be Python overhead"*	**low** — no flashjet changes needed
2	**Improve GPU utilisation**	the profile says most of the GPU is idle; retuning how work is spread should give more speed for free	medium
3	**Test full-event clustering**	so far this is **jet** clustering. Clustering a whole event is a harder, slower problem and has not been re-checked	medium
4	**Other algorithms & conditions**	only anti-k_t is timed (k_t, C/A untested for speed); no scan over jet radius; no test vs **pile-up**	medium
5	**Use it inside the experiment's software**	the real production test. Needs the GPU code rewritten in C++ — a separate project	**high**

> **Recommended next step: item 1.** It is quick, needs no new code, and it closes the one genuine weakness in the numbers presented here.

Backup

Reproducing this

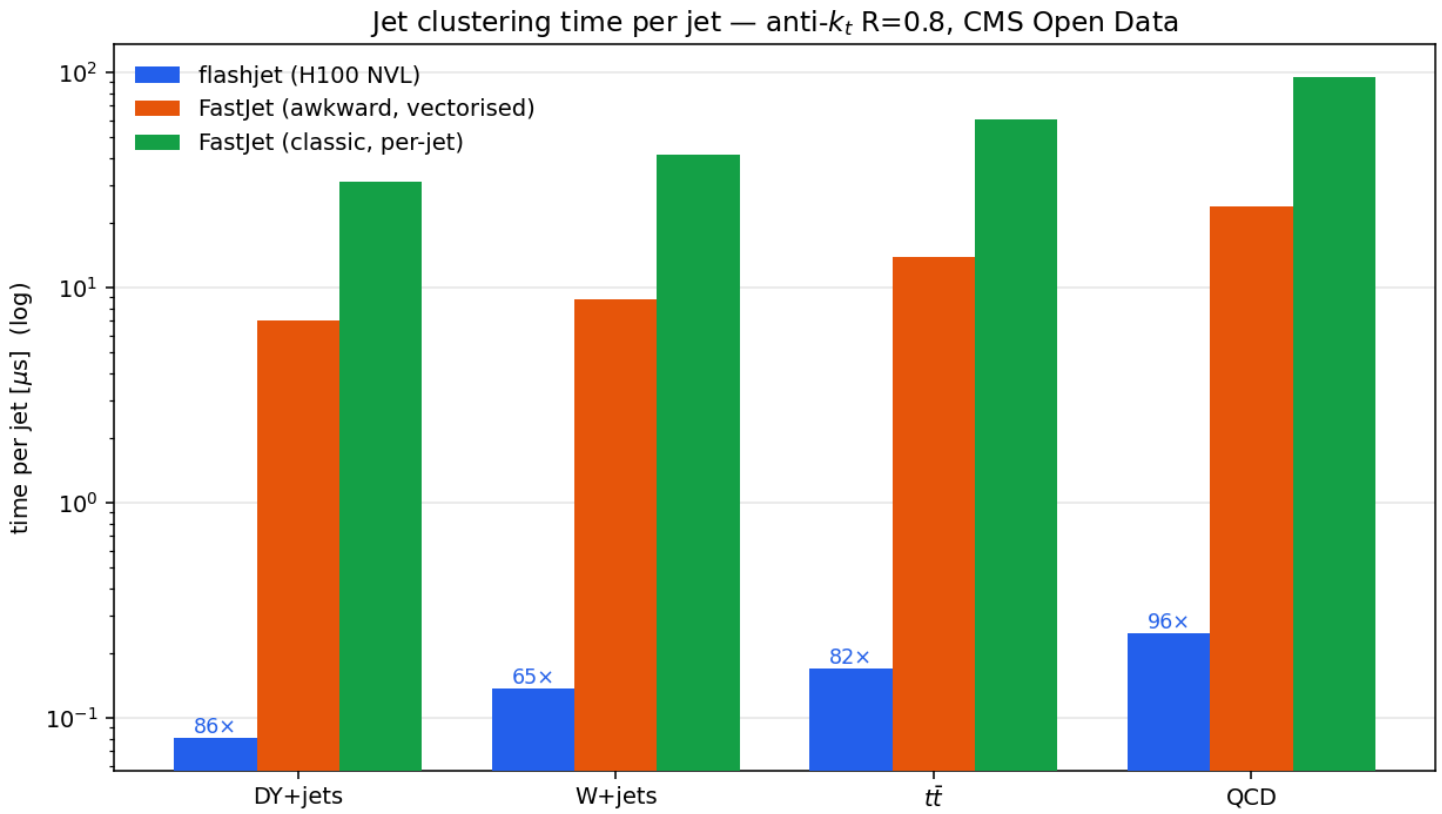
```
# 1. public data -> per-jet particle lists
python3 dump_opendata_constituents.py {ttbar|qcd|wjets|dyjets} 30000 ak4

# 2. timing + profiling on a GPU (batch job)
condor_submit bench_a100.sub      # or bench_h100.sub

# 3. plots
python3 make_bench_plots.py 0.4 H100_NVL
```

Datasets (CMS Open Data, RunIISummer20UL16 MINIAOD):

process	dataset
$t\bar{t}$	`TTToHadronic_TuneCP5_13TeV-powheg-pythia8`
QCD	`QCD_Pt_470to600_TuneCP5_13TeV_pythia8`
W+jets	`WJetsToLNu_1J_TuneCP5_13TeV-amcatnloFXFX-pythia8`
DY+jets	`DYJetsToLL_M-50_TuneCP5_13TeV-madgraphMLM-pythia8`

Read directly over the network from public CERN storage — no grid certificate required.

R = 0.8 — the same conclusion



Anti-k_t with the larger jet radius R=0.8: **65–96×** over vectorised FastJet. The result does not depend on the choice of jet radius.

Detailed profiling numbers

Hardware counters for the clustering step (H100, one representative workload):

metric	value	interpretation
memory (DRAM) throughput	0.02 %	not memory-limited
compute throughput	15.4 %	not compute-limited either
registers per thread	168	high — this is what limits how many threads run at once
theoretical occupancy	18.75 %	ceiling set by the above
achieved occupancy	6.25 %	actual
waves per multiprocessor	0.32	the GPU is not filled even once

Reading: the limitation is **parallelism**, not memory or arithmetic. Reducing the per-thread resource usage, and re-tuning the thresholds that decide how work is split across the GPU, are the concrete levers.

Measured without fixed clock frequencies, so absolute timings here are indicative; the occupancy and register figures are structural and unaffected.